2026 INTERVIEW GUIDE

30 MCP Interview Questions

The July 2026 spec rewrote the protocol. These are the questions interviews ask now.

One question per page · the complete spoken answer · current to the stateless core, Tasks, and CIMD auth.

Fundamentals | Building Servers | Transport | Security | Production

Naved Khan
Senior Gen-AI Engineer

READ THIS FIRST

MCP just shipped its biggest change ever. Most guides are now outdated.

The July 2026 spec made the core stateless, formalized extensions and Tasks, hardened auth with CIMD, and deprecated features every old tutorial still teaches. Interviews reward the candidates who know what changed, and why.

“What is MCP, in plain words?”

THE FULL ANSWER

- An open standard that connects AI applications to tools and data through **one shared interface**.
- A server exposes capabilities once. Any client — Claude, a custom agent, an IDE — can use them. **Build the integration once, every model benefits.**
- The analogy that lands: **USB-C for AI tools**. One port, many devices.
- In 2026 it's the default enterprise wiring for agents, with a formal spec, registry, and versioning policy behind it.

“Walk me through the three roles: host, client, server.”

THE FULL ANSWER

- **Host**: the AI application the user actually touches — a chat app, an IDE, an agent runtime.
- **Client**: the connector the host runs, one per server connection. It speaks the protocol.
- **Server**: the program exposing tools, resources, and prompts — your code, wrapping your systems.
- Why it matters in interviews: the split explains the security model. The host decides **what the model may do**; the server only offers.

“Tools, resources, prompts — what's the difference?”

THE FULL ANSWER

- **Tools**: actions the model can invoke. Functions with typed inputs — "create_ticket", "query_db".
- **Resources**: data the application can read into context — files, tables, documents. Passive, addressable by URI.
- **Prompts**: reusable templates the server offers — "summarize this incident" with slots.
- The senior distinction: tools are **model-controlled**, resources are **app-controlled**. That boundary is a design decision, not trivia.

“Trace one tool call end to end.”

THE FULL ANSWER

- The model decides to call a tool and emits a structured request. The **client sends it as JSON-RPC** to the server.
- The server runs your function and returns the result — now with **structured output** support, so responses can be typed, not just text.
- The client hands the result back into the model's context, and generation continues.
- The security sentence: **the model never executes anything**. It asks; your code acts.

“Why a protocol at all? I can just write function-calling code.”

THE FULL ANSWER

- Function calling is per-app plumbing: N apps × M tools = **N×M integrations**. MCP makes it N+M.
- Discovery is built in: a client asks a server what it offers, at runtime. No hardcoded catalogs.
- And the ecosystem compounds: a registry of servers, shared auth patterns, versioned capabilities.
- Concede the honest case: for **one app with three internal tools**, plain function calling is simpler. Standards pay off at scale.

“MCP vs OpenAI function calling — same thing?”

THE FULL ANSWER

- Related layers, not rivals. **Function calling** is how a model emits a structured call. **MCP** is how applications discover and connect to the tools themselves.
- Most MCP clients use function calling under the hood to drive tool selection.
- One is a model capability; the other is **infrastructure between applications**.
- This distinction is a reported screening question — mixing the layers reads as tutorial-level knowledge.

“Build me a minimal MCP server. What does it look like?”

THE FULL ANSWER

- In Python, FastMCP is the standard path: **a decorator turns a function into a tool**, and the docstring becomes its description.
- The description is not a comment — **the model reads it like documentation** to decide when to call the tool.
- Typed parameters become the schema automatically.
- Say the discipline: one narrow job per tool, informative errors, and names a model can't confuse.

```
from fastmcp import FastMCP
mcp = FastMCP("support-server")

@mcp.tool()
def get_ticket(ticket_id: str) → dict:
    """Fetch one support ticket by its exact ID."""
```

“What makes a good tool description?”

THE FULL ANSWER

- Written for the model, not for humans: **when to use it, what it returns, what it must not be used for**.
- Bad descriptions are the number one silent failure: the model can't pick what it can't understand.
- Keep the catalog small: **ten sharp tools beat forty vague ones** — both for token cost and picking accuracy.
- Treat descriptions as versioned artifacts: a description edit changes model behavior, so it ships like code.

“What is structured tool output, and why did it matter?”

THE FULL ANSWER

- Before it, tools returned text and clients parsed hopefully. Now a tool declares an **output schema**, and results come back typed.
- That kills a whole failure class: downstream code no longer regexes JSON out of prose.
- It also enables chaining: one tool's typed output feeds the next tool's typed input safely.
- Date it to show currency: structured output landed in the **2025-06 revision**, alongside elicitation.

“What is elicitation?”

THE FULL ANSWER

- A server asking the **user** for more input mid-operation, through the client: "which environment — staging or prod?"
- Before it, servers had to guess or fail. Now clarification is a protocol feature with user consent in the loop.
- Design rule: elicit for **decisions**, never for secrets — credentials flow through auth, not chat.
- It's the protocol's version of a junior engineer asking a good question instead of guessing.

“A tool call fails — timeout, bad input, downstream 500. What should the server do?”

THE FULL ANSWER

- Return a **useful error the model can act on**: what failed, why, and what to try instead. "Ticket not found; IDs look like TCK-12345."
- Transient failures: the server retries internally with backoff before surfacing anything.
- Never silently swallow, and never leak stack traces — errors are **part of the tool's interface**.
- The model will read your error text and adapt. Write it like you're prompting, because you are.

“How do you test an MCP server?”

THE FULL ANSWER

- Unit-test the tools as plain functions — they are plain functions.
- Protocol-level: drive it with a test client, assert on **schemas, error shapes, and list results**.
- Then eval-level: does a real model **pick the right tool** for realistic requests? Tool-selection accuracy is measurable.
- The Inspector tool covers manual poking; CI covers regressions. Both, not either.

“stdio vs HTTP transport — when each?”

THE FULL ANSWER

- **stdio**: the client launches the server as a local process. Perfect for personal tools, IDEs, anything on the user's machine.
- **Streamable HTTP**: the server runs remotely, serves many clients, sits behind normal web infra — auth, load balancers, logging.
- Local developer tool → stdio. Team or enterprise service → HTTP.
- Bonus point: SSE as a separate transport is legacy now; streamable HTTP absorbed it.

“The 2026 spec made MCP stateless at the core. Why is that a big deal?”

THE FULL ANSWER

- The old protocol assumed a **long-lived session** — painful behind load balancers and serverless, where any request may hit any replica.
- The **2026-07-28 spec** made the core stateless with header-based routing: any server instance can answer any request.
- That's what makes MCP deployable like a normal web service: **horizontally scaled, cacheable, resilient**.
- Citing this release, by date, is the currency check for 2026 MCP interviews.

“What are Tasks in MCP?”

THE FULL ANSWER

- The answer to **long-running work**: a tool that takes minutes returns a task handle instead of blocking the connection.
- The client polls **tasks/get** for status and results; tasks/update reports progress.
- In 2026 Tasks live in a formal **extension** — out of the experimental core, versioned on their own.
- Use it for report generation, batch jobs, deep research — anything past a request timeout.

```
# long job: return a handle, poll for completion
result = client.call_tool("generate_report", args)
status = client.tasks_get(result.task_id)
```

“What's the extensions framework, and why does it exist?”

THE FULL ANSWER

- A formal way to add capabilities **without bloating the core**: namespaced extensions like io.modelcontextprotocol/tasks, independently versioned.
- Clients and servers negotiate which extensions they both speak.
- It's how the protocol grows without breaking every implementation — plus a **formal deprecation policy** for what leaves the core.
- The pattern to name: small stable core, innovation at the edges. Every protocol that survived did this.

“What got deprecated in 2026, and what do you say about it?”

THE FULL ANSWER

- **Roots, Sampling, and Logging** left the core; **Dynamic Client Registration** is deprecated in favor of CIMD.
- The interview move isn't listing them — it's the reasoning: features barely used, or security models that didn't survive contact with the enterprise.
- Deprecated-but-working matters for real systems: there's a **migration window**, not a cliff.
- Knowing what died signals you run MCP in production, not just tutorials.

“One agent, fifteen MCP servers connected. What breaks?”

THE FULL ANSWER

- **The context window**, first: every server's tool catalog competes for tokens, and tool-picking accuracy degrades with catalog size.
- Fixes: tool retrieval (load relevant tools per request), namespacing, and **cacheable list results** from the 2026 spec.
- Then latency: discovery across fifteen servers at startup needs caching too.
- The design answer: agents get **curated toolsets per role**, not a firehose of everything connected.

“How does authentication work for remote MCP servers?”

THE FULL ANSWER

- MCP servers are **OAuth resource servers**: the client brings a token, the server validates audience and scope.
- Resource indicators are required — a token for one server **can't be replayed** against another.
- The 2026 hardening plus **CIMD** (client ID metadata documents) replaced dynamic registration — identity comes from verifiable metadata, not self-assertion.
- Plain-words version: the server checks not just "who are you" but "is this token really meant for me."

“What is tool poisoning?”

THE FULL ANSWER

- A malicious or compromised server shipping **instructions disguised as tool descriptions or results** — injection through the supply chain.
- The model reads descriptions as truth, so a poisoned description can steer behavior anywhere it's installed.
- Defenses: install from a **trusted registry**, pin versions, review descriptions like code, and run untrusted servers sandboxed.
- The one-liner: an MCP server is a **dependency with a voice**. Vet it like one.

“How do you contain what a connected server can do?”

THE FULL ANSWER

- **Least privilege at every layer**: the server gets scoped credentials; the host enforces tool allowlists per agent role.
- Approval gates on writes — payments, deletes, sends — regardless of which server exposes them.
- Egress control on the server's runtime, so a compromised tool can't exfiltrate.
- Say the philosophy: assume any server can be fooled or hostile, and design so **it breaks nothing important**.

```
if tool.name not in ALLOWED[agent.role]: block()
```

“Where do secrets live in an MCP setup?”

THE FULL ANSWER

- **Never in prompts, never in tool descriptions, never in elicitation.** Everything in context can be read back out.
- Server-side: environment or a secrets manager, injected at runtime. Client-side: OAuth tokens, scoped and short-lived.
- Tool results get scrubbed before they enter context — a tool that echoes a connection string just leaked it to the model.
- Audit logs on every tool call close the loop: who asked, what ran, what returned.

“When would you NOT use MCP?”

THE FULL ANSWER

- One application, a handful of internal tools, one team: **direct function calling is simpler** and has fewer moving parts.
- Ultra-low-latency paths where protocol overhead matters.
- And anywhere you can't vet the server supply chain — a standard for connecting is also a standard for **attack surface**.
- Saying no to the standard, with reasons, is the same senior signal as saying no to agents.

“How do you monitor MCP in production?”

THE FULL ANSWER

- **A trace per tool call**: which agent, which server, arguments, latency, tokens, result size.
- Dashboards: tool-call error rates, per-server latency, and **unusual call sequences** — the security signal.
- Version everything: server versions, tool descriptions, spec revisions in play.
- When behavior shifts, the first diff is the tool catalog: **a changed description is a changed prompt**.

“Design MCP for an enterprise: 40 teams, 200 tools. Sketch it.”

THE FULL ANSWER

- An **MCP gateway**: one entry point enforcing auth, allowlists, rate limits, and audit — servers register behind it.
- Teams own their servers like they own microservices; the gateway owns policy.
- Agents get **curated toolsets per role**, resolved at runtime, not the full catalog.
- This mirrors API-gateway history on purpose — say that, and the interviewer knows you've seen this movie before.

“How do you version an MCP server without breaking clients?”

THE FULL ANSWER

- Additive changes are safe: new tools, new optional parameters. **Renames and removals are breaking** — the model's habits break with them.
- Deprecate with overlap: old tool forwards to new, description says "prefer X", removal comes a version later.
- Pin spec revisions explicitly and test against the ones your clients actually speak.
- The subtle one: **changing only a description is a behavior change**. It goes through evals like a prompt edit.

“An agent suddenly picks the wrong tools after a server update. Debug it.”

THE FULL ANSWER

- Diff the **tool catalog** first: descriptions, names, new tools crowding the context. That's the usual culprit.
- Replay the same request against old and new catalogs — tool-selection evals make this a ten-minute answer, not a day of guessing.
- Check list caching: a stale cached catalog on one replica explains "it only fails sometimes."
- Root cause lands in one of three bins: description drift, catalog bloat, or cache skew.

“MCP vs A2A — where does each fit?”

THE FULL ANSWER

- **MCP connects an agent to tools and data.** A2A connects **agents to other agents** across organizations.
- Different layers of the same stack: an agent uses MCP for its hands, A2A for its colleagues.
- They're complementary and increasingly co-deployed in enterprise architectures.
- One sentence: MCP is the tool belt; A2A is the phone line.

“What's on the MCP roadmap that you're watching?”

THE FULL ANSWER

- The **registry maturing** into real discovery infrastructure with verification — that's the supply-chain answer.
- **MCP Apps**: servers shipping interactive UI, not just tools.
- Async and Tasks getting deeper — agents that work for hours need it.
- Answering this shows you follow the spec's blog, not just its tutorials. Name the 2026-07-28 release as the inflection.

“Close it out: why did MCP win?”

THE FULL ANSWER

- Because it solved the **N×M problem** at exactly the moment every company wired agents to everything.
- Because it stayed **boring**: JSON-RPC, OAuth, a small core, formal deprecations. Boring is what enterprises adopt.
- And because it made integrations **compound**: every new server makes every client more capable.
- The closing sentence: "standards don't win by being clever. They win by being **everywhere and predictable** — and MCP is both now."

Preparing for AI interviews?



Naved Khan

Senior Gen-AI Engineer

Full deep-dive decks on **RAG, agents, evals, system design, safety, and context engineering** are on my profile. New guides weekly.